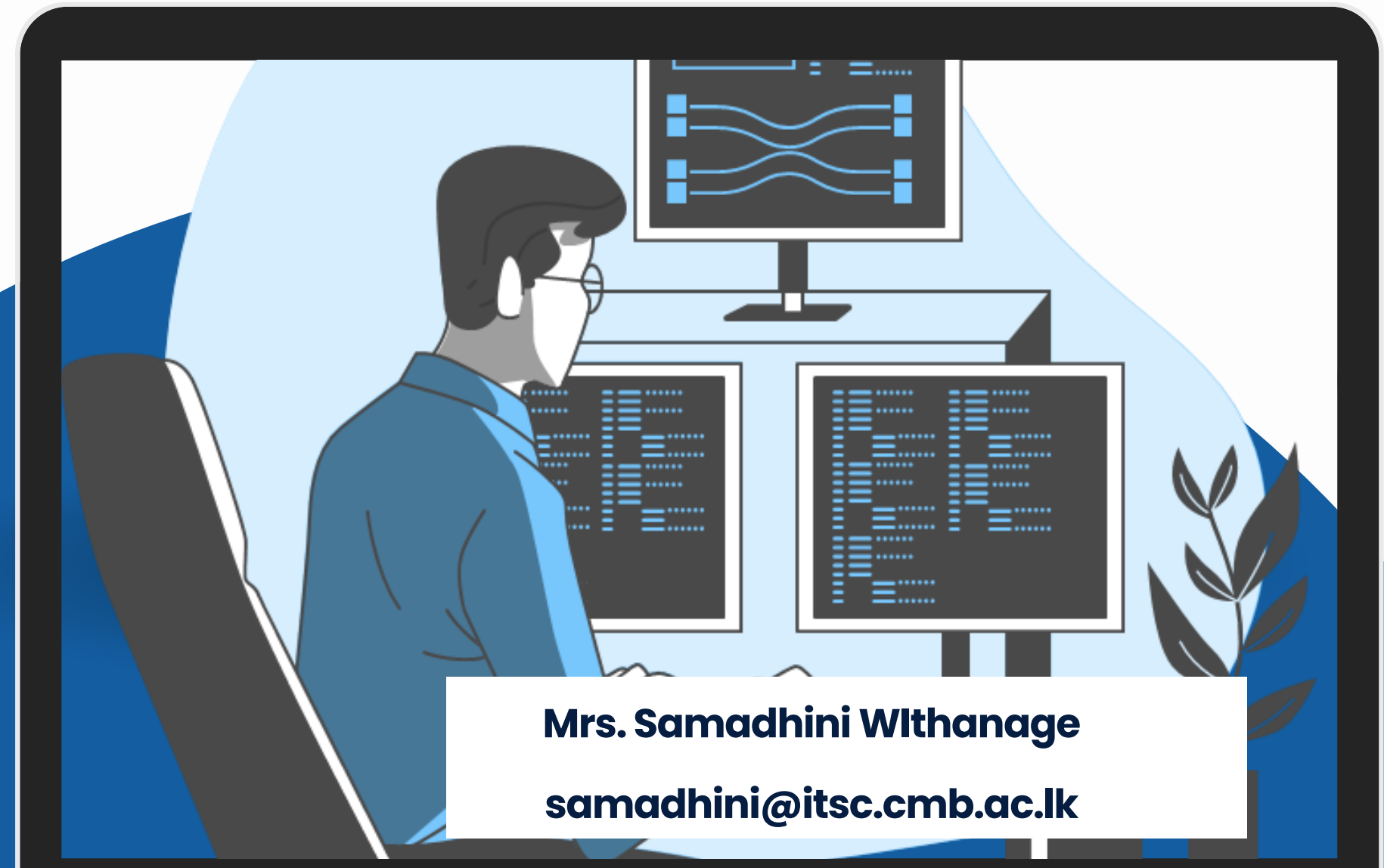


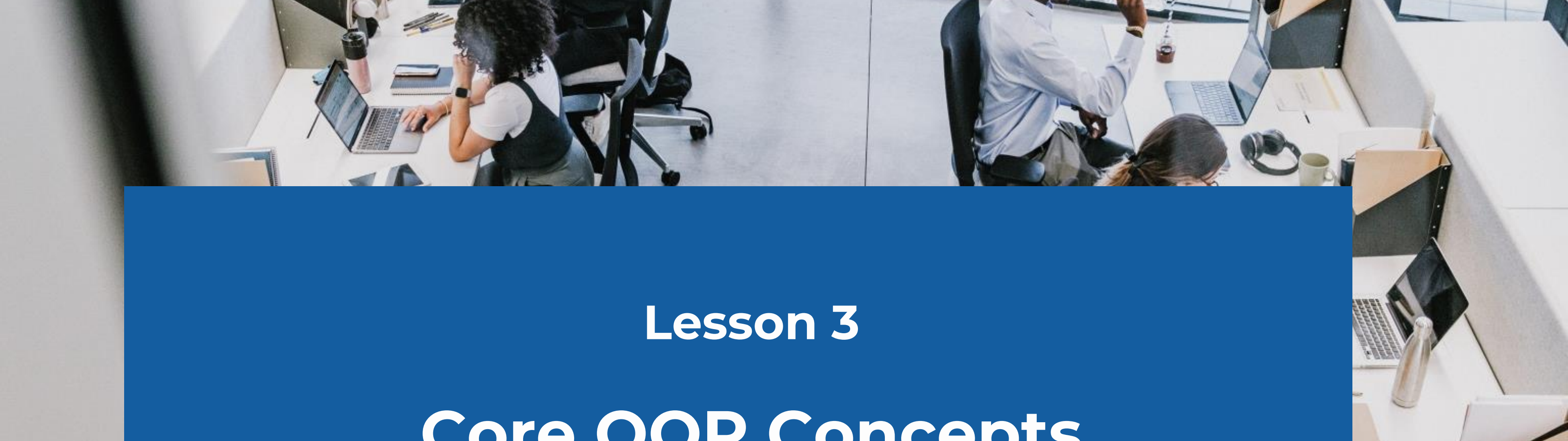
IT 3003

Advanced Programming Techniques



Mrs. Samadhini Wlthanage

samadhini@itsc.cmb.ac.lk

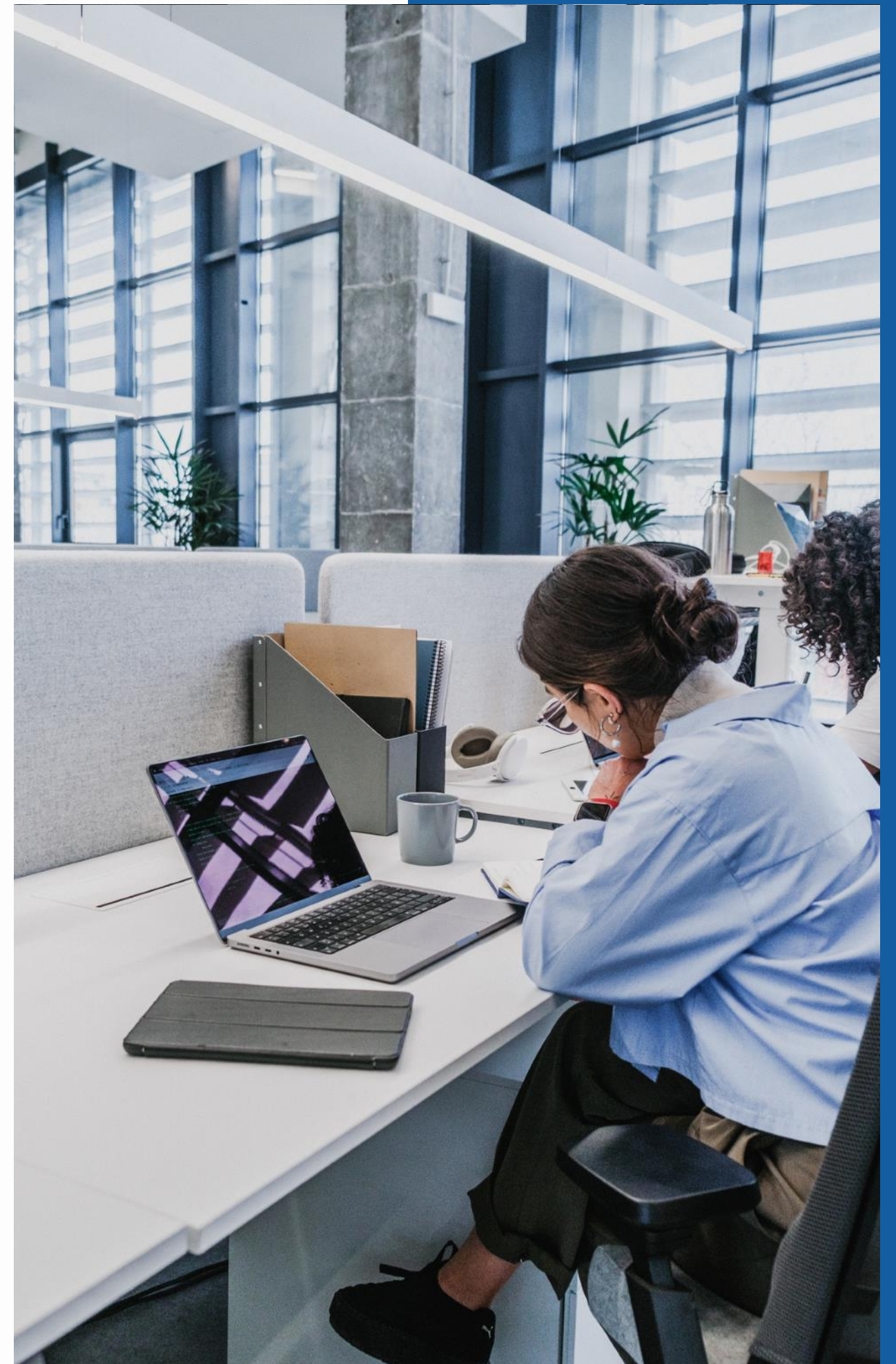


Lesson 3

Core OOP Concepts

Lesson Outline

- ▶ Introduction
- ▶ Classes & Objects (Reminder)
- ▶ Object Lifecycle
- ▶ Constructors
- ▶ Keywords (this , super)
- ▶ Static vs Non-static Context
- ▶ Access Modifiers
- ▶ Core OOP Principles



Introduction

Object-Oriented Programming (OOP) is a programming paradigm



Based on objects and classes

Helps in:



- Code reusability
- Maintainability
- Scalability

Example :

- Calculator app,
- Mobile apps
- ATM systems



Introduction Cont...



Class → Blueprint/template

Object → Instance of a class

Introduction Cont...



Class

A car company designs a model called **Toyota Prius**

The design contains:

color

speed

brand

methods like start(), stop()

But this is only the design — not an actual car.



Object

is the real thing created using the class.

Example:

- Your own red Prius car
- My blue Prius car

Both are objects created from the same blueprint.

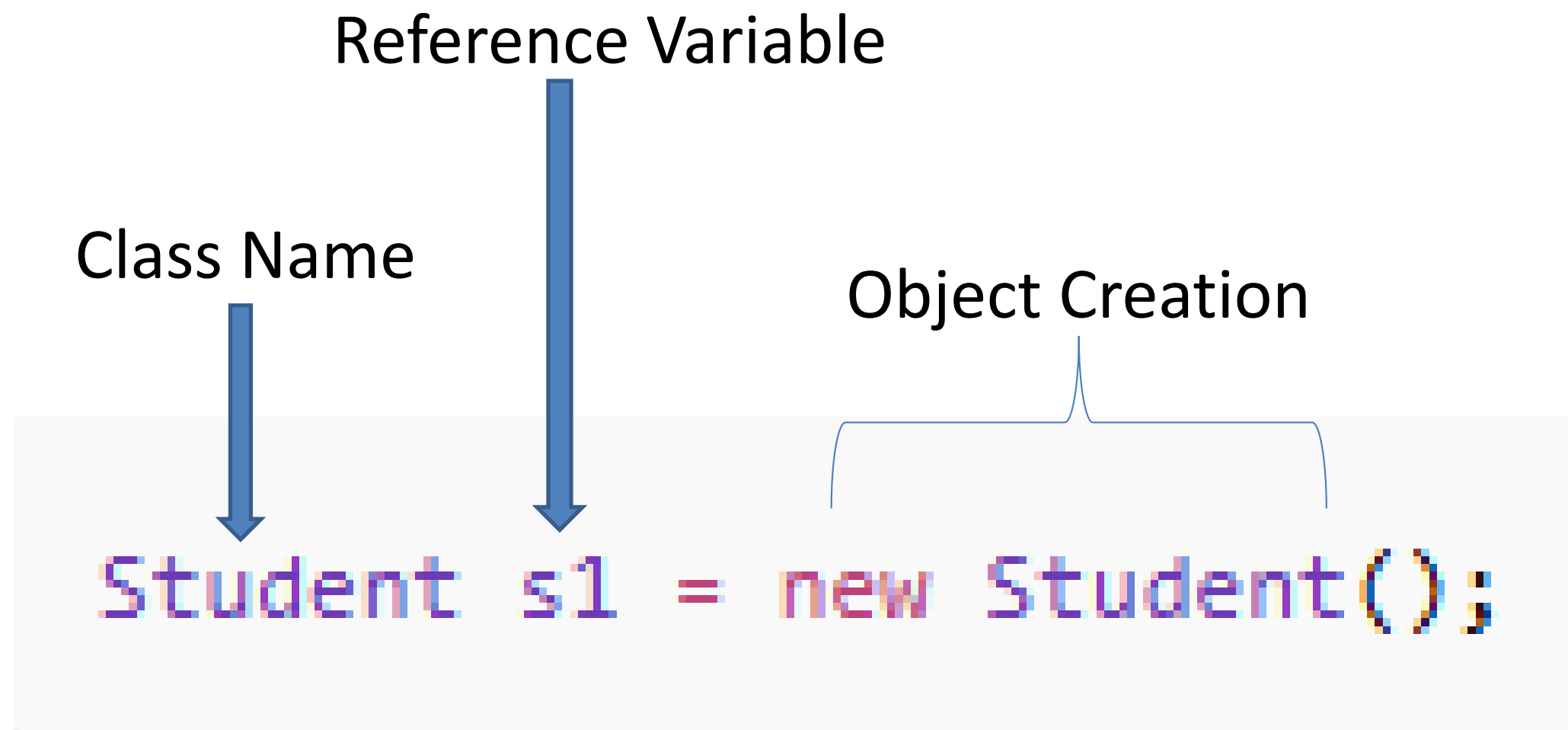
Object Lifecycle

The **Object Lifecycle** describes what happens to an object from the moment it is created until it is removed from memory.

The lifecycle mainly has **4 stages**:

- ▶ Object creation using new keyword
- ▶ Constructor is called
- ▶ Object lives in memory (Heap)
- ▶ Removed by Garbage Collector (GC)

Object Lifecycle Cont..



Object Lifecycle Cont..

Object creation using new keyword

▶ In Java, objects are created using the new keyword.

▶ **What Happens Internally?**

When Java sees new Student():

- JVM allocates memory for the object
- Memory is allocated inside the **Heap Memory**
- Object variables get default values
- Constructor is automatically called

```
</> Java
```

```
Student s1 = new Student();
```

Object Lifecycle Cont..

Constructor is Called

▶ A **constructor** initializes the object.

It runs automatically when the object is created.

```
</> Java

class Student {

    String name;

    // Constructor
    Student() {
        System.out.println("Constructor is called");
        name = "John";
    }

    void display() {
        System.out.println("Student Name: " + name);
    }
}

public class Main {
    public static void main(String[] args) {

        // Object Creation
        Student s1 = new Student();

        s1.display();
    }
}
```

Object Lifecycle Cont..

Object Lives in Heap Memory

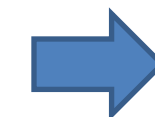
- ▶ All Java objects are stored in **Heap Memory**.

Memory Structure

Memory Area	Purpose
Stack Memory	Stores reference variables
Heap Memory	Stores actual objects

▶ Visualization

Stack Memory	Heap Memory
-----	-----
s1 ----->	Student Object name = "John"



s1 is stored in Stack
Actual object is stored in Heap

Object Lifecycle Cont..

Garbage Collector (GC)



When Does an Object Become Unused?

```
</> Java  
  
s1 = null;
```

- s1 no longer points to the object
- Object becomes unreachable
- JVM marks it for garbage collection

Constructors



A constructor is a special method in Java that is used to initialize objects and is automatically called when an object is created.



When you create an object Java automatically calls the constructor to

- assign initial values
- prepare the object for use



A constructor **must have the same name as the class.**



Because Java uses the class name to:

- identify constructor
- differentiate from normal methods

```
</> Java
```

```
class Student {
```

```
    Student() {    // constructor
```

```
        System.out.println("Student object created")
```

```
    }
```

```
}
```


Constructors Cont..

► Constructors **do NOT** have return type not even void

Incorrect

```
</> Java

class Student {

    void Student() {    // NOT constructor
    }

}
```

correct

```
</> Java

class Student {

    Student() {    // correct constructor
        System.out.println("Hello");
    }

}
```

Why no return type

- Because constructors are NOT called like normal methods.
- Constructor's job is ONLY to create and initialize object, not to return values.

Constructors Cont..

Constructor Overloading

- ▶ Constructor overloading means having multiple constructors in the same class with different parameter lists.

Key Idea : Java allows multiple constructors so that you can **initialize objects in different ways**.

Class Name : Student

```
// Constructor 1 - no data
Student() {
    name = "Unknown";
    age = 0;
}
```

```
// Constructor 2 - only name
Student(String n) {
    name = n;
    age = 0;
}
```

```
// Constructor 3 - name and age
Student(String n, int a) {
    name = n;
    age = a;
}
```

Constructors Cont..

Constructor overloading is based on

- Number of parameters
- Type of parameters
- Order of parameters

Constructors Cont..

Constructor Chaining



Constructor chaining means calling one constructor from another constructor.

This helps:

- reuse initialization code
- avoid duplicate code
- make constructors organized

Constructors Cont..

Before Constructor Chaining

```
class Student {  
  
    String name;  
    int age;  
  
    Student() {  
        name = "Unknown";  
        age = 0;  
    }  
  
    Student(String n) {  
        name = n;  
        age = 0;  
    }  
  
    Student(String n, int a) {  
        name = n;  
        age = a;  
    }  
}
```

```
class Student {  
  
    String name;  
    int age;  
  
    // Constructor 1  
    Student() {  
        name = "Unknown";  
        age = 0;  
    }  
  
    // Constructor 2  
    Student(String n) {  
  
        this();    // calls Constructor 1  
  
        name = n;  
    }  
  
    // Constructor 3  
    Student(String n, int a) {  
  
        this(n);    // calls Constructor 2  
  
        age = a;  
    }  
  
    void display() {  
        System.out.println(name + " - " + age);  
    }  
}
```


Constructors Cont..

Why Constructor Chaining is Important?

- ✓ avoids duplicate code
- ✓ improves readability
- ✓ makes initialization cleaner
- ✓ supports inheritance properly

Announce Quiz

25 Questions

40 min

Upto Core OOP concepts (both theory and practical)

On 05/27

Report

2026/06/ 30

00.00

Keywords (this , super)

this Keyword

this is a **keyword** that **behaves like a reference variable** pointing to the current object.

Mainly Used to:

- Access instance variables

- Call constructors

Keywords (this , super) cont...

Access instance variables

Without using this

```
class Student {  
  
    String name;  
  
    // Constructor  
    Student(String name) {  
  
        // Local variable assigned to itself  
        // Instance variable is NOT updated  
        name = name;  
    }  
  
    void display() {  
        System.out.println("Student Name: " + name);  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        // Creating object  
        Student s1 = new Student("John");  
  
        // Displaying value  
        s1.display();  
    }  
}
```

Output :

Student Name: null

using this

```
class Student {  
  
    String name;  
  
    // Constructor  
    Student(String name) {  
  
        // this.name refers to instance variable  
        this.name = name;  
    }  
  
    void display() {  
        System.out.println("Student Name: " + name);  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Student s1 = new Student("John");  
  
        s1.display();  
    }  
}
```

Output :

Student Name: John

Keywords (this , super) cont...

Call constructors (Constructor Chaining)

```
class Student {  
  
    Student() {  
        System.out.println("Default");  
    }  
  
    Student(String name) {  
  
        this();  
  
        System.out.println(name);  
    }  
}
```

Keywords (this , super) cont...



super Keyword

super is a special keyword in Java that refers to the immediate parent class object.

Main Uses of super

Usage	Purpose
<code>super.variable</code>	Access parent variable
<code>super.method()</code>	Call parent method
<code>super()</code>	Call parent constructor

Keywords (this , super) cont...

▶ Access Parent Class Variable

Without super

```
class Person {  
    String name = "Person";  
}  
  
class Student extends Person {  
    String name = "Student";  
  
    void display() {  
        System.out.println(name);  
    }  
}
```

OUTPUT

Student

With super

```
class Person {  
    String name = "Person";  
}  
  
class Student extends Person {  
    String name = "Student";  
  
    void display() {  
        System.out.println(super.name);  
    }  
}
```

OUTPUT

Person

Keywords (this , super) cont...

▶ Call Parent Class Method With super

```
class Person {  
  
    void show() {  
        System.out.println("Person Method");  
    }  
}  
  
class Student extends Person {  
  
    // Overriding Parent Method  
    void show() {  
        System.out.println("Student Method");  
    }  
  
    void display() {  
  
        // Calling Parent Class Method  
        super.show();  
  
        // Calling Current Class Method  
        this.show();  
    }  
}
```

```
public class Main {  
  
    public static void main(String[] args) {  
  
        // Creating Student Object  
        Student s1 = new Student();  
  
        // Calling display method  
        s1.display();  
    }  
}
```

OUTPUT

```
Person Method  
Student Method
```

Keywords (this , super) cont...

▶ Call Parent Constructor With super

```
class Person {  
  
    // Parent Constructor  
    Person() {  
  
        System.out.println("Person Constructor");  
    }  
}  
  
class Student extends Person {  
  
    // Child Constructor  
    Student() {  
  
        // Calling Parent Constructor  
        super();  
  
        System.out.println("Student Constructor");  
    }  
}
```

```
public class Main {  
  
    public static void main(String[] args) {  
  
        // Creating Student Object  
        Student s1 = new Student();  
    }  
}
```

OUTPUT

```
Person Constructor  
Student Constructor
```



Static vs Non-Static Context

Access Modifiers

- ▶ Access modifiers are keywords used in Java to control the visibility and accessibility of
 - classes,
 - variables,
 - methods,
 - constructors.

- ▶ Types of access modifiers
 - public
 - Private
 - Protected
 - default

Access Modifiers Cont...

private

▶ Accessible only within the same class.

▶ Points

- Most restrictive access modifier
- Cannot be accessed outside the class
- Mainly used for data hiding
- Supports encapsulation

```
class Student {  
  
    private int marks;  
  
}
```

Access Modifiers Cont...

default (No Modifier)

▶ Accessible only within the same package.

▶ Points

- No keyword is written
- Also called package-private
- Cannot be accessed outside the package
- Commonly used for package-level access

```
class Student {  
  
    int marks;  
  
}
```

Access Modifiers Cont...

protected

▶ Accessible within the same package and by subclasses in other packages.

▶ Points

- Supports inheritance
- Child classes can access protected members
- More accessible than default
- Less accessible than public

```
class Person {  
  
    protected String name;  
  
}
```

Access Modifiers Cont...

public

▶ Accessible from anywhere in the program.

▶ Points

- Least restrictive access modifier
- Can be accessed from any class or package
- Used for methods/classes that should be globally available
- Commonly used in APIs

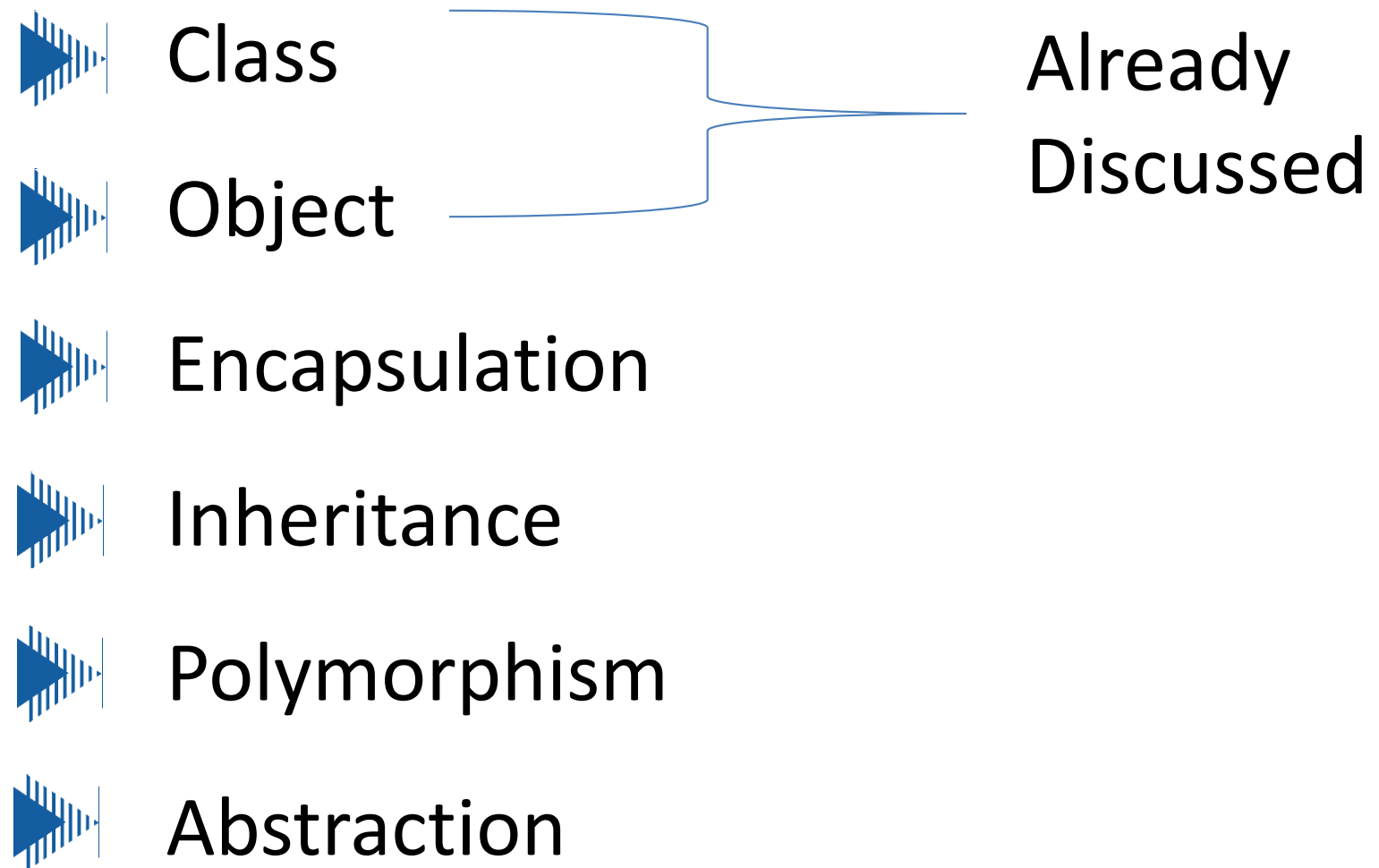
```
public class Student {  
  
    public void display() {  
        System.out.println("Hello");  
    }  
}
```


Access Modifiers Cont...

- private → Best for data hiding
- protected → Mainly used in inheritance
- public → Used for globally accessible members
- default → Used for package-level security

Access Modifier	Same Class	Same Package	Subclass (Other Package)	Other Package
private	✓	✗	✗	✗
default	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

Core OOP Concepts



Core OOP Concepts Cont...

Explanation

▶ Encapsulation is wrapping data and methods into a single unit while hiding data using access modifiers.



Points

- Achieves data
- private variables
- Accessed through getters/setters
- Improves security

```
class Student {  
  
    private int marks;  
  
    public void setMarks(int marks) {  
        this.marks = marks;  
    }  
  
    public int getMarks() {  
        return marks;  
    }  
}
```


Core OOP Concepts Cont...

Explanation

▶ Inheritance allows one class to acquire properties and methods of another class.

▶ Points

- Promotes code reusability
- Uses **extends**
- Supports parent-child relationship

```
class Person {  
  
    void display() {  
        System.out.println("Person");  
    }  
}  
  
class Student extends Person {  
}
```

Core OOP Concepts Cont...

Polymorphism

▶ Polymorphism allows one method to perform different behaviors.

▶ Points

- Means “many forms”
- Improves flexibility
- Two types:
 - Method Overloading
 - Method Overriding

Core OOP Concepts Cont...

Polymorphism cont..

► Method Overloading

Method overloading means creating multiple methods with the same name but different parameters in the same class.

```
class Calculator {  
  
    void add(int a, int b) {  
  
        System.out.println("Sum: " + (a + b));  
    }  
  
    void add(int a, int b, int c) {  
  
        System.out.println("Sum: " + (a + b + c));  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Calculator c = new Calculator();  
  
        c.add(10, 20);  
  
        c.add(10, 20, 30);  
    }  
}
```

Core OOP Concepts Cont...

Polymorphism cont..

Method Overriding

Method overriding means redefining a parent class method in the child class.

```
class Animal {  
  
    void sound() {  
  
        System.out.println("Animal makes sound");  
    }  
}  
  
class Dog extends Animal {  
  
    @Override  
    void sound() {  
  
        System.out.println("Dog barks");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Dog d = new Dog();  
  
        d.sound();  
    }  
}
```

Core OOP Concepts Cont...

Abstraction

▶ Abstraction hides implementation details and shows only essential features.



Points

- Reduces complexity
- Achieved using:
 - Abstract classes
 - Interfaces
- Focuses on “what” not “how”

Core OOP Concepts Cont...

Abstraction cont..

Abstract Class

An abstract class is a class that cannot be instantiated and is used as a base class for other classes.

Points

- Declared using abstract keyword
- Can contain:
 - ✓ Abstract methods
 - ✓ Normal methods
- Can have constructors and variables
- Child classes must implement abstract methods
- Supports partial abstraction

Core OOP Concepts Cont...

Abstraction cont..



Abstract Class

```
abstract class Vehicle {  
  
    abstract void start();  
  
    void stop() {  
  
        System.out.println("Vehicle Stopped");  
    }  
}  
  
class Car extends Vehicle {  
  
    void start() {  
  
        System.out.println("Car Starts");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Car c = new Car();  
  
        c.start();  
  
        c.stop();  
    }  
}
```

Core OOP Concepts Cont...

Abstraction cont..

Interface

An interface is a blueprint that contains abstract methods and is used to achieve full abstraction..

Points

- Declared using **interface** keyword
- Methods are **public and abstract by default**
- Variables are **public static final by default**
- Supports multiple inheritance
- Implemented using **implements**

Core OOP Concepts Cont...

Abstraction cont..



Interface

```
interface Animal {  
  
    void sound();  
}  
  
class Dog implements Animal {  
  
    public void sound() {  
  
        System.out.println("Dog Barks");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Dog d = new Dog();  
  
        d.sound();  
    }  
}
```

Take-Home Exercise

Question 1

Explain the difference between **static context** and **dynamic context in Java**. In your answer, discuss how memory is allocated for static members and instance members, and how they behave during program execution.

Include a real-world example (such as a university system, banking system, or hospital system) to illustrate how static and dynamic contexts are used together in practice.

Also explain:

- Why static variables are shared across all objects
- Why instance variables are unique to each object
- When static methods should NOT be used

Take-Home Exercise cont..

Question 2

Compare **composition and inheritance** in Java object-oriented design.

Your answer should explain:

- The meaning of “is-a” relationship and “has-a” relationship
- How inheritance promotes code reuse but may reduce flexibility
- Why composition is often preferred in modern software design

Discuss a real-world scenario (such as an e-learning platform, car system, or employee management system) where both approaches could be used.

Finally explain:

- Situations where inheritance is appropriate
- Situations where composition is a better design choice
- The concept of tight coupling vs loose coupling in this context

Take-Home Exercise cont..

Question 3

Explain the role of constructors in Java object creation and describe the object lifecycle from creation to initialization.

Your answer should include:

- What happens when an object is created using the new keyword
- The difference between default constructor and parameterized constructor
- The purpose of constructor chaining

Also explain:

- The role of this keyword in resolving ambiguity and referring to current object
- The role of super keyword in accessing parent class constructor and members
- The order of constructor execution in inheritance (parent → child)

Support your explanation with a simple real-world analogy (e.g., building a house, employee hiring process, etc.).

Summary

- ▶ Introduction
- ▶ Classes & Objects (Reminder)
- ▶ Object Lifecycle
- ▶ Constructors
- ▶ Keywords (this , super)
- ▶ Static vs Non-static Context
- ▶ Access Modifiers
- ▶ Core OOP Principles

The background is a blurred office scene with people working at desks, overlaid with a semi-transparent blue filter. A white rectangular box is centered on the image, containing the text "ANY QUESTIONS ?" in a bold, white, sans-serif font. The word "ANY" is on the top line, and "QUESTIONS ?" is on the bottom line.

**ANY
QUESTIONS ?**



THANK YOU!

